已有成果

后端：

- **知识抽取**：把故障树领域知识编码进 RAG 抽取层：三级事件（顶/中间/基本）+ `resultsIn` / `jointly_resultsIn` 表达 OR/AND 门语义，并初步构建了一套prompt。
- **构建知识图谱**：「KG 投影为主、LLM 兜底为辅」的双保险架构，主路径确定性、可溯源、零额外 LLM 成本。
- **构建三元组**：边方向重建启发式（BFS 距离 + 事件类型优先级）、三层降级 JSON 解析，鲁棒性好。

前端：

- **编辑**：前端故障树编辑器功能完整（增删改节点/门、撤销重做、导出 PNG/PDF、草稿持久化）。
- **校验**：校验器四类规则（结构/层级/逻辑/KG 一致性）+ LLM 建议（最基本的逻辑校验）
- **已跑通：文档入库 → RAG 检索问答 → 生成三元组 → 编辑 → 校验 的闭环。**

不足之处

算法 / 抽取层面

- **抽取效果**：针对较短文本，抽取以及构建知识图谱、故障树的准确度较高。对于较长文本，效果欠佳。（**可能需要改进LightRAG以及提示词**），这是本项目最关键的环节。
- 两套数据模型不互通：生成端「三元组」vs 校验端「节点-门」，生成结果进不了自己的校验器。
- **节点置信度写死：没有找到一个能够量化故障树节点生成准确程度的方法。
- **没有使用数据库**（考虑使用neo4j）

前端 / 技术实现层面

- **前端流畅度欠佳（操作不够便捷）、美观度不够**
- **端到端程度不够**：「RAG 生成 → 编辑器」靠手动下载/上传 JSON 串接，后端生成结果在前端被丢弃。
- **赛题中要求的AI 专家辅助功能未实现**
- 前后端关系契约不一致（后端只认 `resultsIn` / `jointly_resultsIn`，前端接受任意关系）。
- 工程卫生：孤儿项目、`.bak`、硬编码密钥、7 种包管理，且项目结构不够规范。

我认为最关键需要改进的几个点：
1、核心算法（尤其是针对长文本或者多文档的抽取分析）
2、引入数据库（图数据库、向量数据库、关系数据库）

3、提高前端操作便捷性、流畅性、系统美观度
4、端到端